

Министерство науки и высшего образования Российской Федерации
Пензенский государственный университет
Кафедра «Вычислительная техника»

ОТЧЁТ

По лабораторной работе №4
По курсу «Разработка кроссплатформенных приложений»
на тему «Работа с файлами»

Выполнили
студенты группы 23ВВИ2:

Родионов А. А.
Пырков Д. А.

Приняли:
Юрова О.В.
Карамышева Н.С.

Пенза 2026

Цель работы: изучить работу с файлами и механизмы сериализации данных.

Лабораторное задание: модифицировать приложение из предыдущей лабораторной работы, реализовав сохранение в файл и загрузку данных из файла. Предусмотреть сохранение данных, как в текстовом виде, так и в двоичном (с использованием механизма сериализации). Для этого нужно добавить 4 кнопки для сохранения и загрузки в текстовом и двоичном виде соответственно. Кроме того, в программе нужно предусмотреть использование стандартного диалога открытия файла (JFileChooser).

Ход работы

Описание работы программы

В среде NetBeans были созданы классы `FileWriter` и `FileReader`. Оба класса содержат исключительно статические методы и не предполагают создания экземпляров. Каждый класс предоставляет два метода — для работы с текстовым и двоичным форматом соответственно. Взаимодействие с пользователем при выборе файла осуществляется через стандартный диалог `JFileChooser`.

Класс `FileWriter`

Метод `saveText`

Метод `saveText(File file, LinkedList<RecIntegral> dataList)` выполняет сохранение коллекции записей в текстовый файл. Для записи используется `BufferedWriter`, оборачивающий стандартный `java.io.FileWriter`. Каждый объект `RecIntegral` из переданного списка преобразуется в строку, в которой значения полей `upperLimit`, `lowerLimit`, `step` и `result` разделены символом точки с запятой. Если поле `result` не было вычислено и содержит значение `null`, в файл записывается строковый литерал `"null"`. Каждая запись занимает ровно одну строку файла. По завершении записи поток автоматически закрывается благодаря конструкции `try-with-resources`. В случае ошибки ввода-вывода метод выбрасывает `IOException`.

Метод saveBinary

Метод `saveBinary(File file, LinkedList<RecIntegral> dataList)` выполняет сохранение коллекции записей в двоичный файл с использованием стандартного механизма сериализации Java. Для этого создаётся `ObjectOutputStream`, оборачивающий `BufferedOutputStream` и `FileOutputStream`. Весь список `dataList` записывается в поток единым вызовом метода `writeObject`. Это возможно благодаря тому, что класс `RecIntegral` реализует интерфейс `Serializable`, а класс `LinkedList` является сериализуемым по умолчанию. Поток закрывается автоматически. В случае ошибки ввода-вывода метод выбрасывает `IOException`.

Класс FileReader

Метод loadText

Метод `loadText(File file)` выполняет чтение данных из текстового файла и возвращает `LinkedList<RecIntegral>`. Для чтения используется `BufferedReader`, оборачивающий стандартный `java.io.FileReader`. Файл читается построчно. Пустые строки пропускаются. Каждая непустая строка разбивается по разделителю «точка с запятой», после чего проверяется, что получено ровно четыре фрагмента. Первые три фрагмента преобразуются в значения типа `double` и передаются в конструктор `RecIntegral`, который выполняет валидацию данных. В случае несоответствия формата строки ожидаемой структуре метод выбрасывает `IOException` с указанием номера проблемной строки. При некорректных значениях параметров выбрасывается `InvalidDataException`.

Метод loadBinary

Метод `loadBinary(File file)` выполняет чтение данных из двоичного файла посредством десериализации. Для этого создаётся `ObjectInputStream`, оборачивающий `BufferedInputStream` и `FileInputStream`. Объект считывается из потока вызовом `readObject`, после чего проверяется его принадлежность типу `LinkedList`. В случае успешной проверки объект приводится к типу `LinkedList<RecIntegral>` и возвращается вызывающему коду. Если прочитанный объект не является экземпляром `LinkedList`, выбрасывается `IOException`. Метод

также может выбросить `ClassNotFoundException`, если класс сериализованного объекта не найден в текущем загрузчике классов.

Обработчики кнопок в `mainForm`

Кнопка сохранения в текстовый файл

Обработчик `ButtonSaveTextActionPerformed` создаёт экземпляр `JFileChooser` с заголовком диалога «Сохранить в текстовый файл» и устанавливает фильтр, ограничивающий отображаемые файлы расширением `.txt`. После того как пользователь выбирает путь и подтверждает сохранение, обработчик проверяет наличие расширения `.txt` в имени файла и при его отсутствии добавляет расширение автоматически. Затем вызывается метод `FileWriter.saveText`, которому передаётся выбранный файл и текущий список записей `dataList`. При успешном завершении операции пользователю отображается информационное сообщение. В случае возникновения `IOException` отображается диалог с описанием ошибки.

Кнопка загрузки из текстового файла

Обработчик `ButtonLoadTextActionPerformed` создаёт экземпляр `JFileChooser` с заголовком «Загрузить из текстового файла» и фильтром по расширению `.txt`. После подтверждения выбора файла пользователем вызывается метод `FileReader.loadText`, возвращающий список загруженных записей.

Загружаемые записи перезаписывают данные что до этого находились в таблице: список `dataList` очищается при помощи метода `clear`, после чего в него помещаются все объекты `RecIntegral`, что были получены из файла. Также очищается и таблица `TableMain` с помощью метода `model.setRowCount(0)`. Затем для каждой записи из списка в модель таблицы добавляется новая соответствующая строка со значениями всех четырёх полей добавляется в модель таблицы `TableMain` методом `addRow`. При возникновении `IOException` или `InvalidDataException` пользователю отображается сообщение об ошибке.

Кнопка сохранения в двоичный файл

Обработчик `ButtonSaveBinActionPerformed` функционирует аналогично обработчику сохранения в текстовый файл, с тем отличием, что фильтр `JFileChooser` настроен на расширение `.dat`, а для записи данных вызывается метод `FileWriter.saveBinary`. Автоматическое добавление расширения `.dat` также предусмотрено.

Кнопка загрузки из двоичного файла

Обработчик `ButtonLoadBinActionPerformed` функционирует аналогично обработчику загрузки из текстового файла. Фильтр `JFileChooser` настроен на расширение `.dat`, а для чтения данных вызывается метод `FileReader.loadBinary`. Загруженные записи точно так же заменяют все предыдущие данные в таблице. В блоке обработки исключений перехватываются `IOException` и `ClassNotFoundException`.

Форматы хранения данных

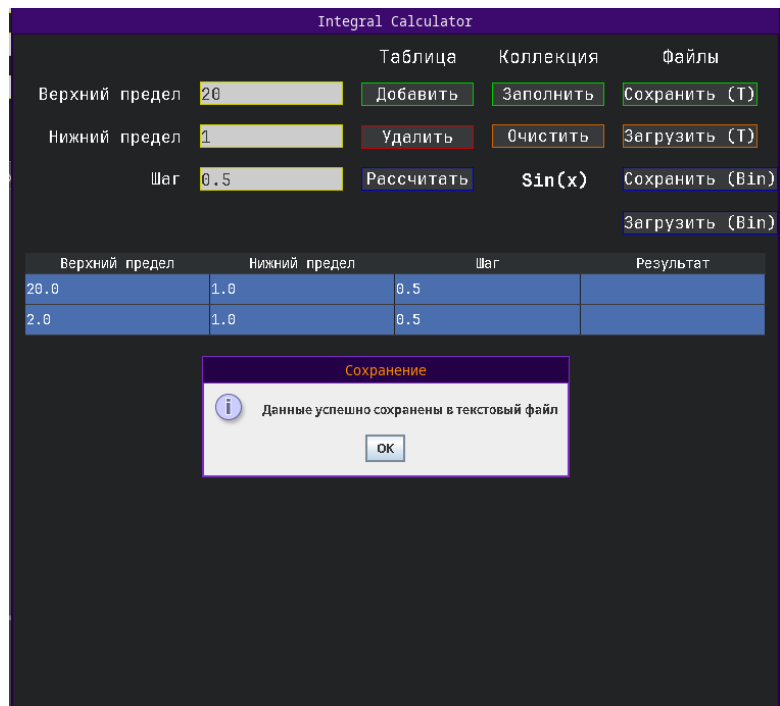
Текстовый формат представляет собой последовательность строк, каждая из которых содержит четыре значения, разделённых символом точки с запятой. Пример строки: `10.0;1.0;0.5;1.8390715290764525`. Данный формат пригоден для чтения и редактирования человеком, а также для обмена данными с другими приложениями.

Двоичный формат представляет собой стандартный поток сериализации Java, содержащий объект `LinkedList<RecIntegral>`. Данный формат обеспечивает точное сохранение состояния всех объектов, включая значение `null` в поле `result`, однако не предназначен для чтения или редактирования вне среды Java.

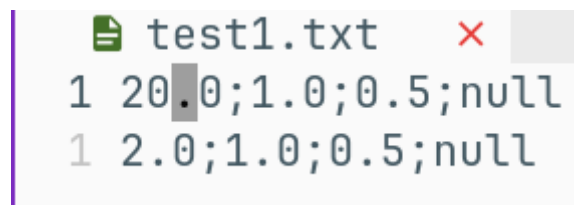
Тестирование программы

Проведём тестирование программы.

Сохраним результат в текстовый файл `test1.txt`

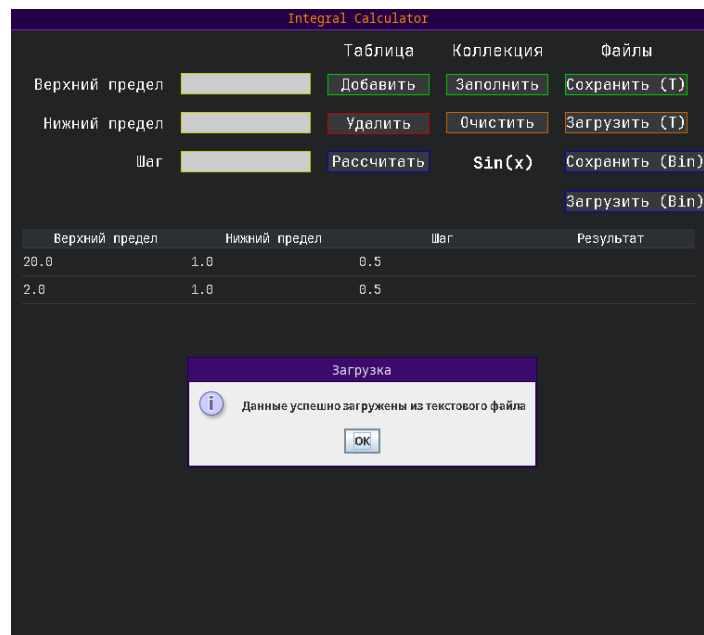


Посмотрим содержимое файла test1.txt.



Содержимое полностью соответствует введенным данным.

Далее выполним загрузку файла.

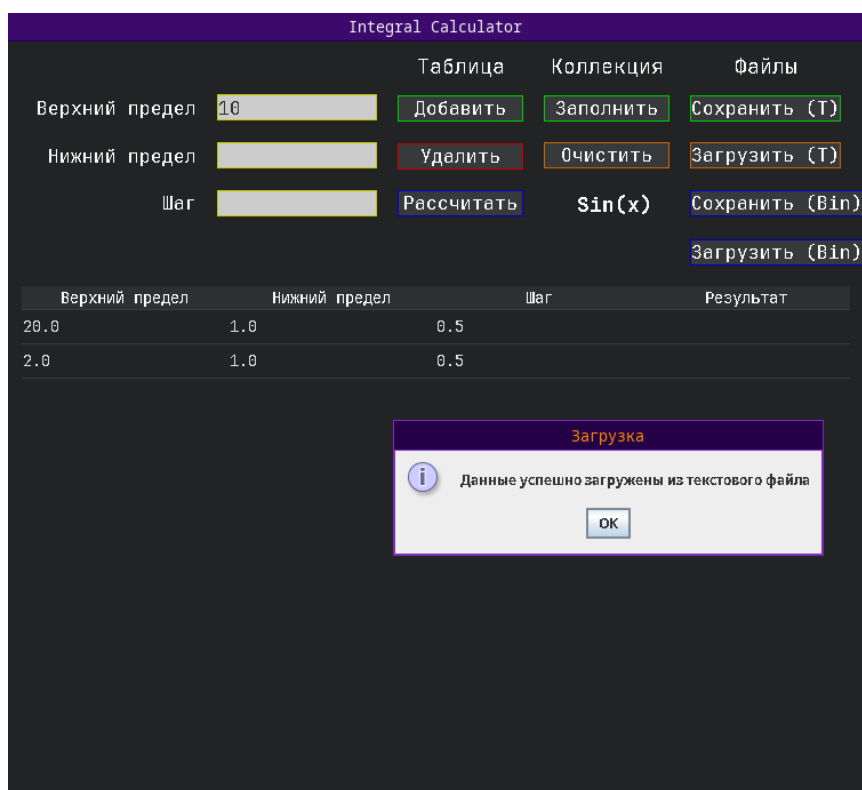


Результат соответствует тому, что было сохранено.

Далее изменим содержимое файла и добавим пустые строки.

```
test1.txt
4 20.0;1.0;0.5;null
3 2.0;1.0;0.5;null
2
1
5
```

Повторно выполним загрузку файла.

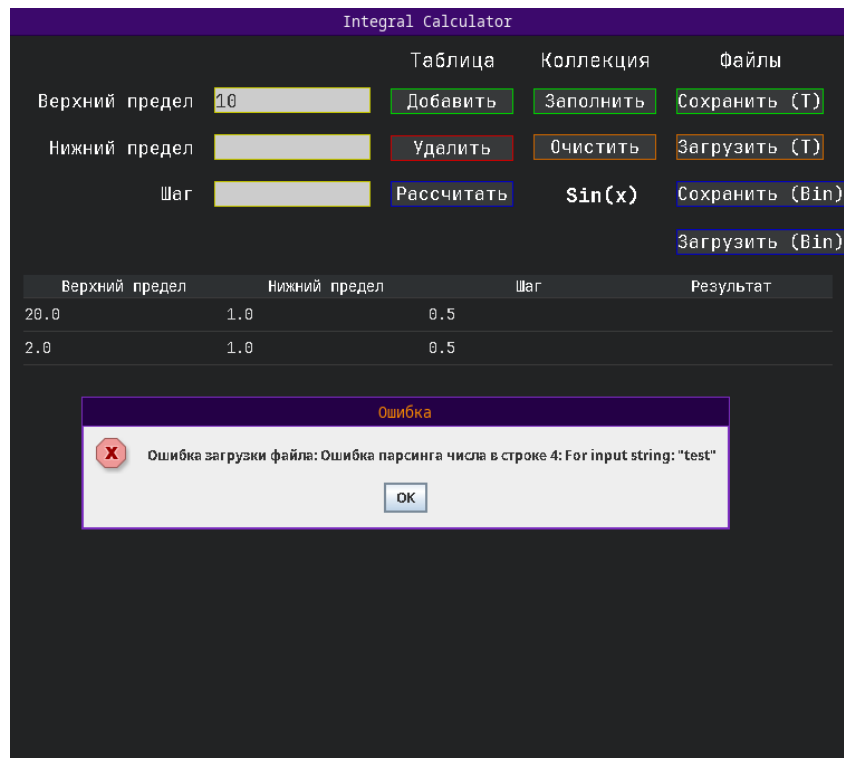


Пустые строки не оказали никакого влияния на работу программы.

Далее намеренно изменим одну из пустых строк таким образом, чтобы данные были не корректными.

```
test1.txt
3 20.0;1.0;0.5;null
2 2.0;1.0;0.5;null
1
4 test;test;test;null
1
```

Теперь выполним чтение этого файла.



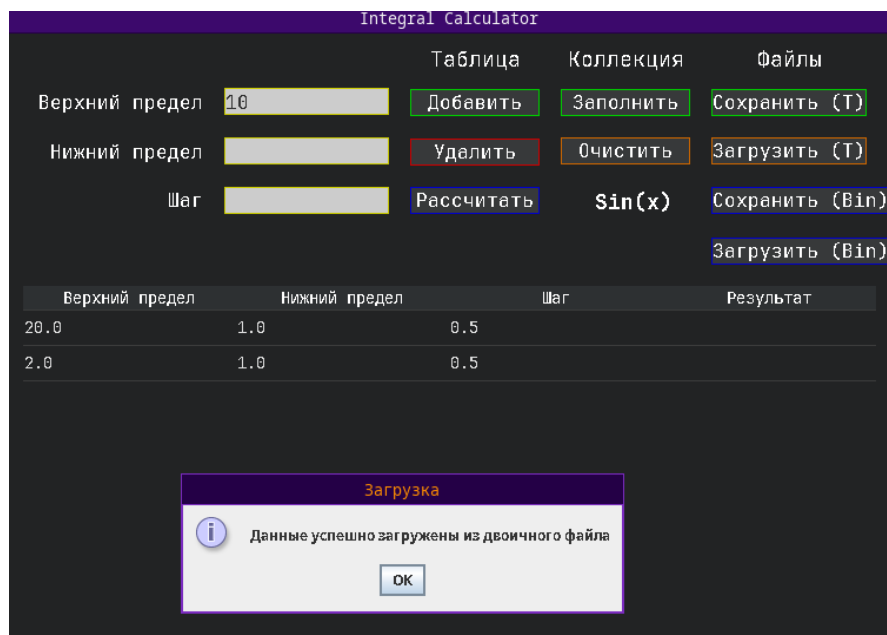
Программа выдала исключение, что является правильным результатом.

Далее выполним сохранение в бинарном формате под названием test2.dat и откроем его в текстовом редакторе.



Как видно файл содержит данные в бинарном формате.

Далее выполним загрузку этого файла.



Содержимое файла загрузилось в том же виде.

Тестирование завершено, программа работает корректно.

Вывод: в ходе выполнения лабораторной работы были получены навыки работы с файлами и механизмами сериализации данных.